Diagnosing Undetected Quantum Mutants: Equivalence, Input Contracts and Oracle Statistics

Seojin Choi (ORCID: 0009-0001-9906-7349)[1] and Geunseok Yang (ORCID: 0000-0001-5677-5129)[1*]

[1]Department of Computer Science and Engineering, Hankyong National University, Anseong, Republic of Korea.

*Corresponding author(s). E-mail(s): gsyang@hknu.ac.kr;
Contributing authors: insui12@hknu.ac.kr;

Abstract

Surviving quantum mutants can preserve behaviour, affect forbidden inputs or evade summary statistics; detection counts alone cannot distinguish these causes. We contribute a census separating these causes in 120 QuCheck files (77 behaviour groups) and 1,136 Muskit files (608 groups); a group shares whole-circuit behaviour up to global phase. The executor contract allows every basis bit string, whereas a workspace mask lists initially zero wires used to store intermediate states. After seeing earlier outcomes, we defined a slot-consistent contract: for each program, its audited upper-register mask fixes the fewest wires to zero while preserving every input preparation. Muskit's allowed-input witness class falls from 130 files in 95 groups to 8 in 7, while 621 files remain detected. Of QuCheck's 50 survivors, 9 differ only outside documented program input restrictions; the other 41 have equivalent fixed builds. Pearson histogram tests expose all 50 summary-limited files in 31 groups among Muskit's 515 survivors, whose histograms differ but summaries agree. Fresh-shot search uses two arms: S gives each test full shots; B shares the original test-catalogue budget across the search. S detects 118 of 130 Muskit executor files; B's shots per test cannot reach either rejection threshold under that contract, whereas both arms detect 8 of 8 slot-consistent files with cumulative histogram tests. Testers should check allowed inputs and full histograms before interpreting survival as equivalence; exploratory findings concern archived small noiseless builds.

Keywords: quantum software testing, mutation testing, input contracts, test oracles, circuit equivalence

1 Introduction

When a quantum mutant survives testing, should a tester change the input, change the oracle's statistics, or expect the same behaviour from that build? A missed source edit may preserve the circuit, affect only inputs forbidden by an input contract specifying allowed inputs and initial register states, or change correlations that the oracle discards. A detection count merges these causes, and circuit-equivalence checks address only the first.

We reuse mutant archives from QuCheck and Muskit, tools for quantum mutation testing; each *subject* is 1 provided mutant, compared with its given unmutated program, the *release control*. Our *test catalogue* is the complete set of fixed differential tests applied to each subject; a *suite* is a subset of those tests. A *slot* fixes circuit arguments or an input preparation, such as flipping the lowest wire before the program runs; each test pairs 1 slot with 1 output measurement basis, Z, X or Y. A *silent* subject survives every test in that test catalogue; a *census* classifies every subject by the cause of its detection or survival. Our *fixed wrapper* adds fixed preparations and common differential tests to archived circuits; each resulting whole circuit is a *build*. A registered preparation is the fixed state preparation specified by a slot; a behaviour group contains subjects from the same program whose full builds agree up to global phase in every slot within the numerical tolerance. The shot-based differential oracle compares single-qubit probabilities and whole-register odd parity, the *existing statistics*; a full histogram retains all measured output bitstrings.

RQ1: Why do mutants survive the complete test catalogue? We distinguish unchanged whole circuits, input-contract limits, phases that depend on input and information hidden by statistics or measurement bases. The *registered input* is the all-zero state before a build; the *application input* is the register state after slot preparation, before the archived program operations (the program body). The *executor*, or circuit runner, admits all computational-basis bit strings under the *executor contract*; workspace wires store intermediate states; *fixed-zero* means assumed initially in $|0\rangle$. A *workspace contract* makes this assumption for the wires listed in a *mask*. A mask is *slot-consistent* when it fixes no wire varied by a slot or by an input preparation documented in source or properties before the program body. The *program-semantic contract* allows only inputs satisfying restrictions documented in source, properties or specifications. The *admissible span* contains all linear combinations of allowed basis states; a coherent input superposes such states with definite relative phases. Figure 1 shows the census order.

For example, Little Bernstein–Vazirani (LittleBV) `Add102.py` inserts a controlled-Z gate between wires q_2 and q_5. At slot 0, the fixed preparation leaves every wire zero and hides the edit. Changing the application input to $|4\rangle$ sets only q_2; the Z probabilities of $q_5 = 1$ become approximately $\{0, 1\}$ in some order, with absolute difference 1.000000. The registered existing-statistic gap is only 2.22×10^{-16}, below the detection threshold, so a surviving edit can need an input change; Section 3 checks whether that change obeys each contract.

RQ2: Which changes to tests expose silent subjects? An exact law is the output probability distribution calculated from the circuit rather than estimated from shots; we distinguish deterministic *exact-law witnesses*, whose statistic gaps exceed

Table 1 Reading key for the census. I groups A, C and P; the other letters are separate classes.

Class	Name	What differs from the release control?
E	Equivalent build	No full-circuit difference up to global phase.
A	Admissible-input witness	An allowed basis input changes the output state.
C	Contract-limited	All states in the admissible span agree up to one global phase; only forbidden inputs differ.
P	Phase-only	Each allowed basis output agrees up to phase; phases between inputs differ.
O	Summary-limited	A full histogram differs, but existing statistics hide it.
M	Measurement-basis-limited	States differ, but the registered bases hide it.
T	Threshold-limited	A statistic gap is positive but below the detection threshold.
D	Detected	An exact registered statistic gap exceeds the detection threshold.

I denotes input-limited: registered output states agree but another input changes them. Formal numerical criteria and counts appear in Table 3.

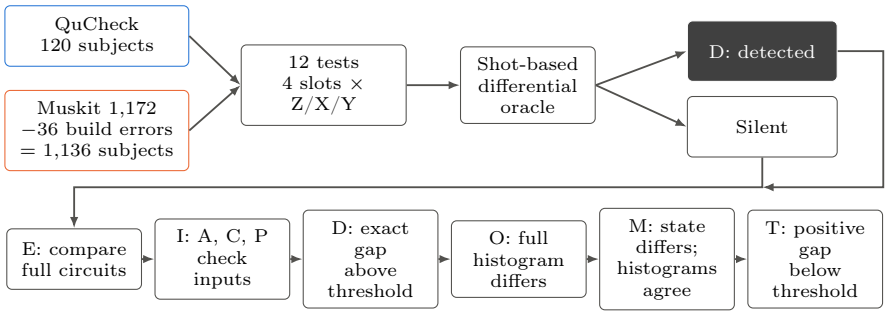


Fig. 1 Both detected and silent subjects enter the numerical census in Algorithm 1's order, E, I, D, O, M, T; the first matching condition assigns the class. Bottom-chain arrows mean "otherwise, try the next condition". Census D uses exact probabilities; the upper oracle uses sampled counts. The reading key defines the letters.

a specified threshold, from fresh-shot detections using new measurement samples. Search-wide error adjustment divides each subject's error allowance α across every comparison in its input search; we compare allowed input changes, richer statistics and wider measurement bases.

The contribution is a contract-conditional explanation of survival, paired with the shot cost of exposing differences by changing inputs or statistics. Muskit A changes from 130 to 8 files under the two contracts, while registered detections stay fixed; Algorithm 1 distinguishes missing inputs from lost correlation information.

2 Background and Related Work

A quantum test does not read out an entire state in 1 shot; each shot yields a bitstring, and repeated shots give a histogram of how often each string occurs. An oracle can then keep a few statistics of that histogram or compare the full histogram. Quantum test systems already address inputs, output probabilities and suite cost; we ask why mutants survive in their fixed test catalogues. QSharpCheck generates constrained inputs and checks Q# properties (Honarvar, Mousavi, & Nagarajan, 2020), while QuCheck adds Qiskit generators, assertions and preconditions (Pontolillo, Mousavi, & Grzesiuk, 2025, 2026). The inspected QuCheck preprint describes X/Y/Z tests with 1-qubit marginals, statistical decisions, multiplicity correction and duplicate removal, though we do not assume its code exactly matches the journal version (Pontolillo et al., 2025, Sections 4.4.1–4.4.3). Statistical assertions check expected state patterns (Huang & Martonosi, 2019), whereas MorphQ tests Qiskit through metamorphic links between program outputs (Paltenghi & Pradel, 2023); a survey reviews these approaches (Paltenghi & Pradel, 2024).

Quito uses input, output and joint coverage with wrong-output and specified-probability oracles, tracking faults found and execution cost; its value-based criteria exclude phase testing (Ali, Arcaini, Wang, & Yue, 2021). MutTG searches for suites that trade off size against mutants found, using the current correct program for expected outputs and matching search counts for random search (Wang, Yu, Arcaini, Yue, & Ali, 2022). Our exhaustive census of these builds under explicit contracts separates survival causes from gains due to selecting more tests in the test catalogue. We connect equivalence calculations to allowed-input and statistic limitations, then measure the shot cost of exposing those differences.

Classical work studies test/resource trade-offs (Yoo & Harman, 2012) and size/overlap threats in mutation testing (Papadakis et al., 2019); quantum-testing guidance also calls for fair baselines and clear settings (Li, Shao, Zhao, & Wang, 2026). Fortunato, Campos and Abreu discuss equivalent mutants and oracle choice in a Qiskit case study (Fortunato, Campos, & Abreu, 2022a); our census separates input-contract and statistic limits within supplied builds. These concerns motivate the study, while Muskit and QMutPy provide quantum mutation systems (Fortunato, Campos, & Abreu, 2022b; Mendiluze, Ali, Arcaini, & Yue, 2021); all mutants here come from the pinned QuCheck and Muskit archives, with none generated for this study or selected by detection outcome. Mutant survival varies with edit type, location and program traits (Mendiluze Usandizaga, Ali, Yue, & Arcaini, 2025), so we report both source-file counts and counts of circuits with the same behaviour, together with source-edit classes.

Circuit-equivalence checkers address semantic questions beyond finite test outcomes (Burgholzer & Wille, 2021). Transpiler-Based Equivalence conservatively identifies full-circuit equivalence when matched transpilation produces identical OpenQASM, assuming a semantics-preserving transpiler; its experimental state-vector reference instead compares 1 shared input state (Campos & Miranskyy, 2026). Our full-unitary calculation checks the supplied wrapper builds, while the state calculation checks outputs from the registered input, the all-zero state before the build runs.

These standard calculations support the joint census and contract comparison without supplying an independent program specification; real-fault overlap with mutation sources measures representation rather than detection (Saad et al., 2026), leaving this oracle's real-fault performance unmeasured. Online Resource 1, Table S4 gives the full comparison and an archive pointer to the source-record inventory.

3 Study Design

3.1 Definitions and input contracts

A *mutant set* contains the subjects checked together; Muskit supplies Conditional Execution (CE), Little Bernstein–Vazirani (LittleBV) and quantum random-access memory (QRAM). For compact ranges, $q_{a:b} = 0$ means every wire from q_a through q_b, inclusive, is fixed zero. The 9 audited masks are executor (none), CE lower/upper, LittleBV upper, QRAM lower/upper, and the joint upper/upper/lower, upper/upper/upper, lower/upper/lower masks in CE/LittleBV/QRAM order. Lower fixes CE $q_{0:2}$ or QRAM $q_{0:1}$; upper fixes CE/LittleBV $q_{3:5}$ or QRAM $q_{3:6}$. A named single-program mask leaves the other programs unrestricted; each joint mask combines the stated wire lists. For each program, the slot-consistent contract uses the audited upper-register mask with the fewest fixed wires preserving every preparation: CE/LittleBV $q_{3:5} = 0$ and QRAM $q_{3:6} = 0$. This choice leaves $q_{0:2}$ available as a common input interface; fixing LittleBV q_2 would also preserve its slots but impose an additional, undocumented restriction. The audit enumerates admitted slots and reclassifies every subject; these assumptions do not recover a unique semantic specification (Online Resource 1, Table S46). An *admissible input* obeys the chosen contract; a *contract-violating state* does not.

The historical *inherited suite* is a subset of tests selected on QuCheck's *development set*: 60 Fourier-transform and phase-estimation subjects. The separate *evaluation set* contains 120 subjects from Deutsch–Jozsa, Grover, teleportation and superdense coding; RQ1/RQ2 use the complete test catalogue, without selecting this suite. The last pair share Bell preparation and form a 60-subject *lineage*, meaning programs that share source code, whereas an algorithm family groups implementations of the same algorithm; no lineage crosses the development/evaluation split. Each test has index $t = 3\ell + b$, where ℓ indexes the slot and $b = 0, 1, 2$ denotes Z, X, Y; Z adds no output rotation, X adds H, and Y adds $S^\dagger$ then H; † denotes conjugate transpose.

Muskit's slots apply X to each set bit of the integers 0,1,2,3 before the program body; for application input $|x\rangle$ after slot ℓ, the input before the build is $|x \text{ XOR } \ell\rangle$; integer kets denote complete registers in little-endian order. Only q_0, q_1 vary in its registered preparations; the broader executor search varies every basis input; we write q_r for qubit r (source code uses `q[r]`); x_0, x_1 and y are input bits and an oracle target bit. Here F_{DJ} is a Boolean oracle, $\oplus$ denotes XOR, and c_1, c_2 are classical message bits. With $i^2 = -1$, $|+\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$ and $|+i\rangle = (|0\rangle + i|1\rangle)/\sqrt{2}$. H, S and X mean Hadamard, $\mathrm{diag}(1, i)$ and bit flip; basis labels X/Y/Z name measurement axes. QuCheck's executor-contract comparison calculates state vectors for additional inputs without shot sampling.

Table 2 Slots and program-semantic input contracts, with an executor contract for Muskit. The registered input is all-zero before build execution; fixed registers constrain alternative inputs.

Program (qubits)	Slot parameters	Admissible domain; fixed initialization
Deutsch–Jozsa (3)	$F_{\mathrm{DJ}} = 0, 1, x_0, x_0 \oplus x_1$ in slot order	No quantum input; $q_{0:2} = 0$; 4 of 8 oracles; oracle $\lvert x, y\rangle \mapsto \lvert x, y \oplus F_{\mathrm{DJ}}(x)\rangle$.
Grover (3)	Marked integer 0,1,2,3 in $q_1 q_0$ order; 1 iteration	No quantum input; $q_{0:2} = 0$, with q_2 the ancilla.
Teleportation (3)	$q_0 = \lvert 0\rangle, \lvert 1\rangle, \lvert +\rangle, \lvert +i\rangle$	Arbitrary data state on q_0; $q_1 = q_2 = 0$; coherent corrections (quantum-controlled corrections).
Superdense coding (2)	Classical messages $c_1 c_2 = 00, 01, 10, 11$	These 4 messages; $q_0 = q_1 = 0$ before encoding.
Fourier transform (4)	0; X on q_0; H on all; H then S on all	Full data register, no fixed ancilla; development only.
Phase estimation (4)	Phases $0, 1/8, 2/8, 3/8$ turn (a full rotation as the phase unit)	Unitary/eigenstate arguments; estimation $q_{0:2} = 0$, target $q_3 = \lvert 1\rangle$; development only.
CE (6)	Common integer prefix; internal X/H/P (phase-gate) preparation and uncomputation remain inside the body, unchanged by the slot	Executor contract: all basis strings on $q_{0:5}$; no documented workspace restriction.
LittleBV (6)	Common prefix varies q_0, q_1; q_2 stays 0 during registered prefix preparation	Executor contract: all basis strings on $q_{0:5}$; no documented workspace restriction.
QRAM (7)	Common prefix; internal address H/P/H and memory-bank routing remain inside the body	Executor contract: all basis strings on $q_{0:6}$; no documented workspace restriction.

QuCheck contracts follow source properties and builders; Muskit uses the executor contract because no program-semantic restriction is documented. The slot-consistent contract fixes CE and LittleBV q_3–q_5 and QRAM q_3–q_6 to zero without excluding slots; both contracts appear in Table 3. Source evidence and Muskit input preparation: Online Resource 1, Table S24 and Online Resource 1, Table S13.

3.2 Shot decisions and numerical census

Each subject and release control uses $N = 8{,}192$ shots per test; for subject m and test t, let $\hat{p}_j$ and $\hat{q}_j$ estimate statistic j in their measured histograms. With comparison allowance K and per-subject error allowance α, the threshold and detection indicator are

$$D(m,t) = \mathbf{1}[\hat{d}_{mt} > \tau(K,N)], \quad \hat{d}_{mt} = \max_j |\hat{p}_j - \hat{q}_j|, \quad \tau(K,N) = \sqrt{2\ln(4K/\alpha)/N}. \tag{1}$$

Here **1** is one when its predicate holds and zero otherwise; the registered comparison uses $K = 120$, $\alpha = 0.001$ and $\tau_0 = 0.056513$. Here K counts allowed statistic comparisons. Baseline replay repeats the baseline test with the same inputs and seed to check stored results; the historical allowance uses $(12 + 1)(8 + 1) = 117$: tests plus replay, each with an 8-qubit cap and parity. The cap is conservative: these builds use at most 7 qubits; rounding to 120 adds slack rather than another measured comparison (Online Resource 1, Table S33). Its per-subject Bernoulli union bound is $4Ke^{-N\tau^2/2}$; it does not control errors across all subjects. QuCheck reuses a release-control histogram among subjects run together for the same program and test; Muskit reuses 1 per program and test; records with the same seed are replays, not independent control draws.

For slot ℓ, $U_{m,\ell}$ and $U_{c,\ell}$ are subject and release-control unitaries on n qubits. Let $u_\ell = |1 - |\operatorname{tr}(U_{m,\ell}^\dagger U_{c,\ell})|/2^n|$ measure unitary difference; the trace sums diagonal entries. Registered output states $\psi_{m,\ell}, \psi_{c,\ell}$ have infidelity $f_\ell = 1 - |\langle\psi_{m,\ell}|\psi_{c,\ell}\rangle|^2$; for exact probabilities, $d_{\ell b} = \max_j |p_j - q_j|$; hats on d denote shot estimates throughout both resources. Write $u = \max_\ell u_\ell$, $f = \max_\ell f_\ell$ and $d = \max_{\ell,b} d_{\ell b}$, with registered maxima over admitted slots. Table 3 gives the numerical tolerances and every class letter; a resolved gap exceeds ϵ_o, the tolerance separating a probability difference from numerical noise. A *numerical state witness* records a state difference above ϵ_s; an *exact-law witness* instead exceeds a specified statistic threshold using exact probabilities. A state difference defines A; a statistic gap need not exceed the detection threshold, so A and exact-law witness counts can differ. The groups are fixed before contract changes and do not count independent defect mechanisms.

Ties at a detection threshold count as non-detections; admissible-input search exhausts all allowed slots and basis inputs without stopping at its first witness. Only illustrative contract-violating states use a first-gap scan, trying basis states before pairwise superpositions; teleportation checks the full allowed data subspace (Online Resource 1, Table S24).

Silent subjects through the census.

The Introduction's LittleBV `Add102.py` has $u = 0.500000 > \epsilon_u$ but registered $f = 0.000000$ and $d = 2.22 \times 10^{-16}$ within tolerance: Algorithm 1 assigns I. Its $|4\rangle$ witness sets the unrestricted q_2, making it A under both contracts.

A *pairwise parity observable* is $P(q_r \oplus q_s = 1)$ for $r < s$; the all-pairs comparison includes every such pair.

CE source `000f781fb6a1c07892cb390e` instead has $u = 0.292893$, $f = 0.500000$ and $d = 2.22 \times 10^{-16}$. Its registered state and X histogram differ at slot 0, but existing statistics agree: it is O under both contracts. Pearson histograms expose it; the pair (q_2, q_3) explains the correlation (exact gap 0.500000, sampled 0.498413). Full source IDs and sampled examples are in Online Resource 1, Table S22.

3.3 Histogram comparison and fresh-shot search

The *split-alpha Pearson comparison* gives half of α to existing statistics and half across 12 two-sample Pearson histogram tests, without adding pairs. The existing-statistic

Table 3 Class glossary and Muskit census under both contracts. Counts are files (behaviour groups).

Class	Numerical definition	Executor	Slot-consistent
E	Full builds agree up to global phase.	154 (3)	154 (3)
A	Registered states agree; an admissible basis input changes a state.	130 (95)	8 (7)
C	The full admissible span agrees up to one global phase.	0 (0)	171 (116)
P	Each basis output agrees up to phase; those phases differ, so a coherent input can expose them.	181 (97)	132 (69)
O	Registered full histograms differ; existing statistics agree.	50 (31)	50 (31)
M	Registered states differ; registered full histograms agree.	0 (0)	0 (0)
T	A gap above numerical noise satisfies $\epsilon_o < d \leq \tau_0$.	0 (0)	0 (0)
D	A registered existing-statistic gap exceeds τ_0.	621 (382)	621 (382)
I	Aggregate of A, C and P; no additional subjects.	311 (192)	311 (192)

$\epsilon_u = 10^{-9}$, $\epsilon_s = 10^{-10}$, $\epsilon_o = 10^{-12}$; N = 8,192, $\alpha = 0.001$, $K = 120$, $\tau_0 = 0.056513$. E is checked first; I is subdivided into A/C/P. Both columns total 1,136 files (608 groups), without counting I twice. QuCheck executor counts are E 41 (4), A 2 (2), P 7 (7), D 70 (64); C/O/M/T are 0. Under its program-semantic contract, QuCheck instead has E 41 (4), C 9 (9), D 70 (64).

threshold rises from τ_0 to $\tau_{\text{split}} = \sqrt{2\ln(4K/(\alpha/2))/N}$; the allowance remains $K = 120$ for these builds. For equal-size counts a_z, c_z in fixed output-bitstring bins z, $X^2 = \sum_{z:a_z+c_z>0}(a_z-c_z)^2/(a_z+c_z)$; degrees of freedom equal the number of pooled-positive bins minus 1; with none, $p = 1$. A histogram rejects at $p < \alpha/(2 \cdot 12)$; pooled-zero bins are removed, without selecting bins by signal or pooling sparse bins. Exact-law pseudo-counts Np, Nq give deterministic numerical witnesses, not sampled tests or power estimates; the fresh-shot search instead checks every admissible input, slot and Z/X/Y basis. A role is either the subject or its release control; each statistic uses independent draws for both roles. For H searched histograms, let $L = H[n + 1 + \binom{n}{2}]$ count scalar tests and $J = L + H$ count all elementary tests. With n_{free} unfixed wires, $H = 2^{n_{\text{free}}} \times 4$ slots $\times$ 3 bases. Executor CE/LittleBV use $n_{\text{free}} = 6$, so $H = 768$; QRAM uses $n_{\text{free}} = 7$, so $H = 1{,}536$. All slot-consistent programs use $n_{\text{free}} = 3$, giving $H = 96$. Half of $\alpha = 0.001$ covers the L scalar tests and half covers all H histograms; this is the search-wide adjustment. Thus scalar thresholds use comparison allowance $K = L$, $\alpha/2$ and each test's actual N; histograms reject at $p < \alpha/(2H)$. A rung is a cumulative test family: *Input* uses existing statistics over allowed inputs; *+ pairs* adds every pairwise parity observable; *+ histogram* also adds Pearson histograms. These are the rung names in Table 4; no alpha is spent anew at a rung. An arm is a shot-allocation rule: S uses N = 8,192 per elementary test per role; B divides

Algorithm 1 Census over the registered builds

Construct $U_{m,\ell}, U_{c,\ell}$; compute global u and registered f, d on admitted slots; stop for nonunitary builds.
If $(u \le \epsilon_u \wedge f > \epsilon_s) \vee (f \le \epsilon_s \wedge d > \epsilon_o)$, stop: conflicting diagnostics.
If $u \le \epsilon_u$, assign E.
Else if $f \le \epsilon_s$, assign I: A if an admissible basis output changes beyond a global phase; P if only phases between basis outputs differ; C if the entire admissible span agrees up to one global phase.
Else if $d > \tau_0$, assign D and check sampled $D(m, t)$.
Else if $d \le \epsilon_o$ and a full registered histogram differs, assign O.
Else if $d \le \epsilon_o$, assign M and audit other local bases.
Else assign T.

the test-catalogue budget, $12 \times 8{,}192 = 98{,}304$ samples per role, across all J tests in fixed order. This design samples each elementary test separately, although a practical implementation could share bitstrings among statistics at the same circuit setting and obtain more shots per setting; that different allocation was not evaluated. Every searched test is sampled, with no stopping or control reuse; Bernoulli statistics use the exact two-bin pushforward of the full multinomial law. The archived experiment uses 1,000 repetitions, independent subject streams and recorded PCG64 seeds (Online Resource 1, Table S51). *Per-subject error control* covers each subject's whole search, without dividing α across the subject set; Pearson remains asymptotic, especially with sparse arm-B bins, so it has no exact finite-sample level guarantee. For phase-only sensitivity, 1-qubit $|+\rangle$ inputs and the all-$|+\rangle$ input before each slot extend beyond the executor contract.

Use of AI tools.

We used generative AI tools (Claude Opus 5.5, OpenAI Codex, ChatGPT and Gemini 4 Argon) as follows. For translation, English editing and packaging support, the authors manually reviewed all outputs.

4 Results for RQ1: Why Mutants Survive

4.1 Census and numerical checks

RQ1 answer: survival reflects full-build equivalence, input restrictions or lost histogram information; the admissible-input count depends on the stated contract. Figure 2 shows both executor and slot-consistent contracts, with file and behaviour-group totals in Table 3. Muskit has 1,136 files in 608 groups, including 515 survivors; executor counts are E 154 in 3 groups, A 130 in 95 groups, P 181 in 97 groups, O 50 in 31 groups and D 621 in 382 groups. Under the slot-consistent contract, A becomes 8 files in 7 groups, C 171 in 116 groups and P 132 in 69 groups; E, O and D remain unchanged.

QuCheck has 120 files in 77 groups. Under the executor contract its silent classes are 41 E, 2 A and 7 P; these executor counts are unaffected by the separate Muskit

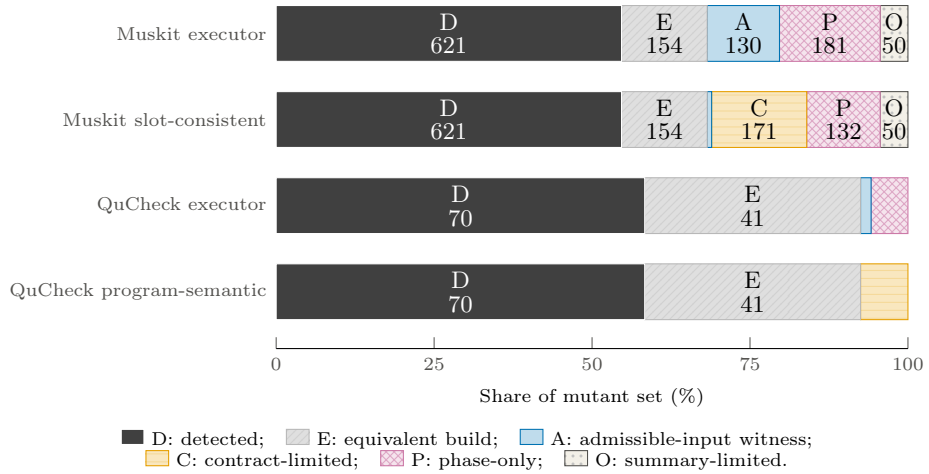


Fig. 2 Contract-conditional census, showing both main Muskit contracts with the same class colours and patterns. Muskit totals 1,136 files (608 groups); QuCheck totals 120 (77). QuCheck's program-semantic contract is shown separately from its executor contract, which is unchanged by the Muskit contract choice. Thin segments: Muskit slot-consistent A 8; QuCheck executor A 2 and P 7; QuCheck program-semantic C 9. Group counts are in Table 3; T and M are 0 at registered inputs.

contract choice. Under program-semantic contract, QuCheck's silent subjects are 41 E and 9 C; this C class concerns only 3-qubit Deutsch–Jozsa, Grover and teleportation builds and 2-qubit superdense coding (Table 3). Here m denotes the numeric suffix of an archived mutant filename. It is not a classification over the native generators' full ranges: Grover subject m8 is detected by the separate native QuCheck property-test run (Section 4.3), while the other 8 C subjects' native non-detections do not prove C across those ranges. The upstream archive declares 40 subjects semantics-preserving; all pass the full-unitary check, and 1 further subject also has equivalent builds.

Exact and saved-shot decisions agree in all 1,440/1,440 QuCheck and 13,632/13,632 Muskit cells (Online Resource 1, Table S1). Neither mutant set has a registered T or M subject; every included build is unitary and none triggers Algorithm 1's conflicting-diagnostics stop. Muskit has 20 positive subthreshold cells from 5 QRAM subjects, each detected in another basis; T is a subject-level class, so these cells add no T subject (Online Resource 1, Table S1). Changing any numerical tolerance by $\times 10$ or $\div 10$ changes no class; raising the shot threshold can move D to T (Online Resource 1, Table S21).

4.2 What the contract changes

Muskit's source documents no fixed-zero workspace or excluded-input rule; example input–output pairs do not establish a wire map or forbid other inputs. The slot-consistent contract changes A from 130 to 8 files under the fixed-wire definitions in Section 3. It changes the permissible alternative inputs without removing a registered detection. The application input is defined just after slot preparation, before the program body; coherent gates inside the body do not change that input domain.

Source and slot audits, with excluded-slot sensitivities, are in Online Resource 1, Table S46.

Under QuCheck's program-semantic contract, no silent subject has an admissible input change that exposes it. The Grover witness $|4\rangle$, for example, changes an ancilla required to start at zero and belongs only to the executor contract. The inserted gate in the contract-limited teleportation file acts on an ancilla already in $|+\rangle$; the build agrees for arbitrary allowed data input. Deutsch–Jozsa's slots cover 4 of the 8 constant or balanced 2-bit oracles. Checking the missing 4 adds no detection: the silent subjects stay 10 E and 3 C under the program-semantic contract (Online Resource 1, Table S19).

4.3 Native QuCheck property-test comparison

A separate CPU execution preserves QuCheck's native generators, preconditions and assertions; its first attempt produced 93 `KeyError` failures in native measurement-ID mapping; these are execution errors and supply no positive or negative detection outcome. A repair protocol was hashed before rerunning with native default circuit deduplication, keeping properties, inputs, seeds and statistics unchanged. The repaired run detects 68/120 subjects and agrees with the fixed-wrapper outcome on 116/120. Native-only detection is Grover m8; catalogue-only detections are Grover m11 and teleportation m9 and m13. Native detections include 0/41 E and 0/4 controls. Total budgets are unmatched and native generators vary qubit counts and other arguments. Both attempts, protocol hashes and counts remain in Online Resource 1, Table S45; their dumps lack the assertion-to-circuit map needed to reconstruct native p-values independently.

5 Results for RQ2: Which Changes Expose Survivors?

5.1 Witnesses, fresh-shot detections and search cost

RQ2 answer: allowed input changes expose additional A subjects, full histograms expose all registered O subjects, and the remaining QRAM differences need mixed measurement axes. Table 4 separates deterministic exact-law witnesses from fresh-shot detections with error adjustment across each subject's entire search; for executor A, the exact-law rungs have 68, 76 and 118 files; arm S reaches the same counts after search-wide adjustment. Table 4 gives their distinct group counts and each contract's own A denominator. Muskit executor B cannot reject at any rung by construction, as shown below; slot-consistent B reaches 6, 8 and 8 of 8 files, and the counts at estimated detection probability $\hat{p} \geq 0.95$ and $\hat{p} \geq 0.5$ coincide in every reported scope. The *set-wide false-positive rate*, labelled "Any E" in the table, counts repetitions flagging at least one E subject. Its pooled scope includes both archives: 41 QuCheck plus 154 Muskit E files; Muskit-only rates are reported separately below.

Table 4 Muskit A: exact-law witnesses and fresh-shot detections under search-wide per-subject error adjustment. Counts are files (groups).

Contract	Rung	Exact-law witnesses	S	B	Pooled Any E, S	Pooled Any E, B
Executor	Input	68 (58)	68 (58)	0 (0)	0.000 (0.000)	0.000 (0.000)
Executor	+ pairs	76 (65)	76 (65)	0 (0)	0.000 (0.000)	0.000 (0.000)
Executor	+ histogram	118 (89)	118 (89)	0 (0)	0.071 (0.005)	0.013 (0.002)
Slot-consistent	Input	8 (7)	8 (7)	6 (5)	0.000 (0.000)	0.000 (0.000)
Slot-consistent	+ pairs	8 (7)	8 (7)	8 (7)	0.000 (0.000)	0.000 (0.000)
Slot-consistent	+ histogram	8 (7)	8 (7)	8 (7)	0.069 (0.002)	0.016 (0.001)

Each contract uses its own A denominator: executor 130 files (95 groups), slot-consistent 8 (7). Arms S and B count $\hat{p} \geq 0.95$; counts for $\hat{p} \geq 0.5$ are identical. Each probability uses 1,000 fresh-shot repetitions. Pooled Any E is the frequency of at least one flag among all 195 E files from both archives (7 group representatives), at each rung; parentheses give the group frequency, not a subgroup file rate. Exact-law witnesses use the historical numerical criteria, not sampled power. Group frequencies test only the fixed smallest-source-ID representative in each group. Allocation sizes and shot ranges are stated in the text and Online Resource 1.

Full subgroup scopes and intervals: Online Resource 1, Table S51 through Online Resource 1, Table S54.

Arm S uses $J \times 8{,}192$ shots per role versus $12 \times 8{,}192$ in the test catalogue: its cost multiplier is $J/12$, with $J = L + H$. Executor CE/LittleBV therefore cost $17{,}664/12 = 1{,}472$ catalogue budgets; QRAM costs $46{,}080/12 = 3{,}840$. For Muskit executor B, $2 \leq N \leq 6$ and

$$\tau_{\text{scalar}}(K, N) = \sqrt{2\ln(4K/(\alpha/2))/N} \geq \sqrt{2\ln(4 \cdot 16{,}896/(\alpha/2))/6} > 1.$$

No probability gap can exceed 1, so no scalar test can reject; for histograms, $(a_z - c_z)^2/(a_z + c_z) \leq a_z + c_z$, so $X^2 \leq 2N \leq 2 \cdot 6$. For any positive degrees of freedom ν, its tail is at least $\Pr\{\chi^2_1 \geq 2 \cdot 6\} > \alpha/(2 \cdot 768)$; with $\nu = 0$, $p = 1$. Thus neither scalar nor histogram tests can reject at the test-catalogue budget; the archived B simulation only checks this analytical consequence (Online Resource 1). The slot-consistent B results remain empirical under their larger per-test allocations.

Specificity and its scope.

Table 4 includes all 195 E files in 7 groups at every rung. A group frequency tests only its fixed, lexicographically smallest source-ID representative, rather than flagging a group when any member is flagged. Both initial rungs have 0 events, while the histogram rung has executor frequencies of 0.071 for S and 0.013 for B, versus 0.069 and 0.016 under the slot-consistent contract. These are set-wide events, not an individual subject's probability or simultaneous control at α across all E subjects. For Muskit alone, executor histogram-rung file frequencies are S 0.052 and B 0.000; slot-consistent frequencies are S 0.059 and B 0.002. The pooled executor B frequency comes entirely from QuCheck's 41 E files, whose smaller searches receive more shots per test. All subgroup and representative-group rates appear in Online Resource 1,

Table S51 through Online Resource 1, Table S54; separate seeds cause QuCheck's frequencies to differ between the two contract comparisons even though its input domain stays fixed. The pooled any-E frequency limits claims of simultaneous specificity but does not change the separately computed A counts. All rates condition on archived noiseless laws; their per-subject error scope is defined in Section 3.

5.2 Information discarded by existing statistics

At registered inputs, the generic split-alpha Pearson comparison exposes all 50 O files in 31 groups, all CE, without pairs. It uses fixed output-bitstring bins and no input-specific bin or pair selection, loses 0 existing detections and flags 0/195 E in saved shots. The pair (q_2, q_3) explains all these recovered correlations in the supplied CE subjects; every other CE pair stays below the original threshold, and the source edits are 48 additions and 2 replacements at 6 gate-call boundaries (Online Resource 1, Table S20). O is defined by a differing histogram, but the above-threshold recovery and its concentration in 1 CE correlation are additional numerical observations, not a general test-change success rate. Saved E non-detection does not establish specificity because release-control records were reused; the independent simulations use separate control draws. The registered split-alpha calibration and the fresh-shot search use different test families; their rates and intervals are reported separately (Online Resource 1, Table S44; Online Resource 1, Table S51).

5.3 Measurement bases and phase-only inputs

The 12 QRAM A subjects remaining after the executor histogram rung ($130 - 118 = 12$) change output states under other inputs while retaining identical full histograms in each homogeneous setting, which uses the same axis on every qubit. To test the missing correlations, a *mixed local measurement setting* chooses X, Y or Z independently on each qubit; *correlation weight* counts non-identity factors in a Pauli string. The all-input negative result covers 128 basis inputs, 4 slots and all $3n + 3^2\binom{n}{2} = 210$ non-identity Pauli statistics of weight at most 2 for $n = 7$. For each residual, an exhaustive 16,384-coefficient Pauli expansion at one fixed existing input finds a weight-4 witness, without stopping at the first gap. This minimum is conditional on that fixed input, not a global minimum across inputs; the all-input weight-at-most-2 negative result is a separate check (Online Resource 1, Table S42).

The phase-only sensitivity uses 1-qubit $|+\rangle$ inputs and the all-$|+\rangle$ input before each slot, distinguishing 181/181 executor P subjects with existing statistics. Those coherent inputs extend beyond the executor contract and can violate the assumed fixed-zero masks; they are not admissible test changes under either main contract.

6 Applying the Diagnosis

The diagnosis orders a tester's decisions: check full-build equivalence, establish the allowed input domain, then distinguish a missing input from lost histogram information or a missing measurement setting. The narrow Muskit prefix and marginal/parity oracle create the laboratory conditions for A, P and O; their shares do not characterize

Muskit's native testing or tools that already compare full histograms. At larger scales, a circuit-equivalence checker can replace the dense-unitary first step, while selected admissible inputs can supply positive witnesses; sampled inputs cannot certify their absence. Such approximations would change the exhaustive scope and require separate validation; the procedure has not been evaluated on other programs or against Quito or MutTG.

7 Discussion

The two RQs separate contract-conditional survival from the cost of detecting differences: executor and slot-consistent contracts preserve registered D while changing A/C/P. The fresh-shot search retains the exact-law counts when each elementary test receives full shots, at the stated cost multipliers; at the test-catalogue budget, Muskit executor B cannot reject by construction.

The historical suite selection chose tests on development subjects; an equal-budget comparator uses the same number of target calls. The Muskit holdout checks (C1–C4) test whether fixed descriptive patterns recur before Muskit outcomes were inspected; their chronology followed inspection of QuCheck outcomes. Historical margin comparisons measure gaps above threshold, while the source-tool outcome cross-tab pairs upstream test outcomes with wrapper classes. None supplies evidence for RQ1 or RQ2; Online Resource 1 retains their definitions, restricted verdicts, unmatched budgets and missing-label limits.

8 Threats to Validity

8.1 Construction, references and numerical scope

Shared builders, compilation and release controls limit what code agreement shows because an incorrect circuit or control can survive both paths. A separately implemented NumPy code path reads circuit text and computes gates, rotations and statistics separately, while sharing supplied builds and control identities. Here separately implemented means distinct parsing and arithmetic under the same study protocol, rather than validation by another team. Marginals, parity and homogeneous measurement bases do not determine the full output state. Full-unitary checks use floating-point tolerances and allow global phase; they do not give symbolic proofs, and the fresh-shot scalar bound covers each subject's entire stated search, not all subjects simultaneously.

8.2 Exposure and mutant-set choices

All census, test-change and sensitivity analyses are post-hoc and exploratory after exposure, meaning earlier outcomes were seen before analysis design; enumeration and internally hashed protocols document ordering without establishing independent validation on programs unseen during design.

Provenance.

The historical QuCheck equal-budget comparator was fixed after earlier outcomes; the Muskit protocol preceded Muskit outcomes but followed QuCheck outcomes. A pre-outcome amendment excluded 36 subjects that failed to build from 1,172 planned subjects, leaving 1,136 buildable subjects. They comprise 10 invalid-qubit-index and 26 invalid-gate-arity exclusions; the planned-denominator analysis remains incomplete because outcomes for the excluded, unbuildable subjects were not analysed. The native execution's failed and corrected attempts have separately hashed protocols. These records preserve scientific history without supplying an independent application specification.

QuCheck's *algorithm families* contain implementations of the same algorithm; shared Bell code contributes 2 times as many files as either other evaluation lineage; source edits, QASM signatures and behaviour groups do not establish independent faults. Muskit's 3 programs share a repository and mutation system; neither mutant set is a validated real-fault sample, and program and hardware differences cannot isolate a hardware effect or predict noisy-device behaviour.

8.3 Execution provenance

We reduced stored counts and circuit outputs without rerunning the original QuCheck simulator; its interpreter patch, binary, driver and firmware remain unpinned. An earlier resampling run's raw records were unavailable for independent reduction, so it supplies no evidence here. Muskit has a pinned environment and preserved execution records; numerical analyses recompute preserved builds (Online Resource 1, Table S29). The binary-dependency inventory distinguishes saved-count reduction and CPU census reconstruction from fresh original-runtime execution (Online Resource 1, Table S55). The former do not require the original QuCheck GPU binary; the latter remains sensitive to its unpinned environment, and no binary restores missing historical records.

The original census used 22.21 seconds and peak resident memory 401,912 KiB for static builds of at most 7 qubits; dense complex128 storage needs 16×4^n bytes per unitary. All 36/36 analytic release-control checks and 4,556/4,556 source-adapter checks pass. CE and QRAM expectations come from their source gates rather than independent application specifications (Online Resource 1, Table S23). The census does not support intermediate measurement, reset, classical control or hardware noise. Native property-based testing changes generated circuit ranges and total budget, so its outcome comparison does not evaluate matched test effectiveness.

9 Conclusion

RQ1 answer: survivors of these supplied builds comprise equivalence, input limitations and lost histogram information; the input contract changes Muskit A from 130 to 8 files without changing registered detections. RQ2 answer: input changes plus full histograms expose 118 executor A files with full shots per test, and generic Pearson testing exposes all 50 registered O files without pair selection. The remaining QRAM A subjects need mixed-axis observations at the examined inputs; Muskit executor B

cannot reject at the test-catalogue budget by construction. A tester should establish allowed inputs and inspect full histograms before interpreting survival as equivalence; these exploratory findings remain limited to the supplied builds and contracts.

Statements and Declarations

Ethics and consent

Not applicable. This study involves no human participants or animals.

Data, Material and/or Code availability

Online Resource 1 is the supporting tables PDF, indexed in its opening resource map; Online Resource 2 is the accompanying replication ZIP, whose paths are relative to its `package/` directory. The archive instructions are at Online Resource 2, `README.md`, with the original recorded-results workflow at Online Resource 2, `reproduce.sh` and the added census workflow at Online Resource 2, `R3_README.md`. The material separates rebuilding saved counts, numerical recomputation of frozen circuits and fresh simulator execution; the original QuCheck interpreter patch, binary, driver and firmware remain unpinned. The manuscript is `QSA_SQJ_manuscript.pdf`. Online Resource 1 is `QSA_SQJ_Online_Resource_1.pdf`; Online Resource 2 is `QSA_SQJ_Online_Resource_2.zip`. The LaTeX source is `QSA_SQJ_latex_source.tar.gz`. Permanent deposition is deferred until acceptance. The four files are also available at https://github.com/insui12/qsa-replication.

Funding

No funding was received for conducting this study.

Competing interests

The authors have no competing interests to declare that are relevant to the content of this article.

Authors' contribution statements

Seojin Choi designed the study, conducted the experiments and analyses, and wrote the manuscript. Geunseok Yang supervised the work and provided critical feedback on the manuscript. All authors read and approved the final manuscript.

Use of AI tools

See the Use of AI tools paragraph in Section 3.

References

Ali, S., Arcaini, P., Wang, X., Yue, T. (2021). Assessing the effectiveness of input and output coverage criteria for testing quantum programs. *2021 14th IEEE*

Conference on Software Testing, Verification and Validation (ICST) (pp. 13–23). Retrieved from https://doi.org/10.1109/ICST49551.2021.00014

Burgholzer, L., & Wille, R. (2021). Advanced equivalence checking for quantum circuits. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, *40*(9), 1810–1824, https://doi.org/10.1109/TCAD.2020.3032630 arXiv:2004.08420

Campos, J., & Miranskyy, A. (2026). *Quantum mutant equivalence via transpilation.* arXiv preprint arXiv:2606.26604. Retrieved from https://doi.org/10.48550/arXiv.2606.26604 (Inspected version: https://arxiv.org/abs/2606.26604v1)

Fortunato, D., Campos, J., Abreu, R. (2022a). Mutation testing of quantum programs: A case study with Qiskit. *IEEE Transactions on Quantum Engineering*, *3*, 1–17, https://doi.org/10.1109/TQE.2022.3195061

Fortunato, D., Campos, J., Abreu, R. (2022b). QMutPy: a mutation testing tool for quantum algorithms and applications in Qiskit. *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis* (pp. 797–800). Retrieved from https://doi.org/10.1145/3533767.3543296

Honarvar, S., Mousavi, M.R., Nagarajan, R. (2020). Property-based testing of quantum programs in Q#. *Proceedings of the IEEE/ACM 42nd International Conference on Software Engineering Workshops* (pp. 430–435). Retrieved from https://doi.org/10.1145/3387940.3391459

Huang, Y., & Martonosi, M. (2019). Statistical assertions for validating patterns and finding bugs in quantum programs. *Proceedings of the 46th International Symposium on Computer Architecture* (pp. 541–553). Retrieved from https://doi.org/10.1145/3307650.3322213 (Inspected version: https://arxiv.org/abs/1905.09721v1)

Li, Y., Shao, M., Zhao, J., Wang, Q. (2026). *A methodological analysis of empirical studies in quantum software testing.* ACM Transactions on Software Engineering and Methodology. Advance online publication. Retrieved from https://doi.org/10.1145/3819590

Mendiluze, E., Ali, S., Arcaini, P., Yue, T. (2021). Muskit: A mutation analysis tool for quantum software testing. *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)* (pp. 1266–1270). Retrieved from https://doi.org/10.1109/ASE51524.2021.9678563

Mendiluze Usandizaga, E., Ali, S., Yue, T., Arcaini, P. (2025). Quantum circuit mutants: Empirical analysis and recommendations. *Empirical Software Engineering*, *30*(4), Article 100, https://doi.org/10.1007/s10664-025-10643-z

Paltenghi, M., & Pradel, M. (2023). MorphQ: Metamorphic testing of the Qiskit quantum computing platform. *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)* (pp. 2413–2424). Retrieved from https://doi.org/10.1109/ICSE48619.2023.00202 (Inspected version: https://arxiv.org/abs/2206.01111v2)

Paltenghi, M., & Pradel, M. (2024). *A survey on testing and analysis of quantum software.* arXiv preprint arXiv:2410.00650. Retrieved from https://doi.org/10.48550/arXiv.2410.00650 (Inspected version: https://arxiv.org/abs/2410.00650v1)

Papadakis, M., Kintis, M., Zhang, J., Jia, Y., Le Traon, Y., Harman, M. (2019). Mutation testing advances: An analysis and survey. *Advances in Computers* (Vol. 112, pp. 275–378). Retrieved from https://doi.org/10.1016/bs.adcom.2018.03.015

Pontolillo, G., Mousavi, M.R., Grzesiuk, M. (2025). *QuCheck: A property-based testing framework for quantum programs in Qiskit.* arXiv preprint arXiv:2503.22641. Retrieved from https://doi.org/10.48550/arXiv.2503.22641 (Inspected version: https://arxiv.org/html/2503.22641v1)

Pontolillo, G., Mousavi, M.R., Grzesiuk, M. (2026). QuCheck: A property-based testing framework for quantum programs in Qiskit. *ACM Transactions on Quantum Computing*, *7*(4), 1–32, https://doi.org/10.1145/3815169

Saad, A.M., Sabit, A.A., Brandenburgh, J., Wang, J., Zhang, Q., Hossain, S.B. (2026). *Are we testing the wrong quantum bugs? a study of fault representation gaps in quantum software testing.* arXiv preprint arXiv:2610.22089. Retrieved from https://doi.org/10.48550/arXiv.2610.22089 (Inspected version: https://arxiv.org/abs/2610.22089v2)

Wang, X., Yu, T., Arcaini, P., Yue, T., Ali, S. (2022). Mutation-based test generation for quantum programs with multi-objective search. *Proceedings of the Genetic and Evolutionary Computation Conference* (pp. 1345–1353). Retrieved from https://doi.org/10.1145/3512290.3528869

Yoo, S., & Harman, M. (2012). Regression testing minimization, selection and prioritization: a survey. *Software Testing, Verification and Reliability*, *22*(2), 67–120, https://doi.org/10.1002/stvr.430